

Expense Tracker — Deployment Guide

Manual steps you need to complete, plus key reference information. Generated from our build session — Phases 14 through 18.

Where We Are

Phases 1–13 are complete: a working local full-stack app (React + Flask + SQLite), tested, documented, and pushed to github.com/v595/expenses. Everything from here on (Phases 14–18) requires you to create accounts on external services — I cannot sign up for these on your behalf. This guide lists exactly what to do at each step and what information to send back to me.

Local Development Quick Reference

Frontend	<code>http://localhost:5173</code>	<code>cd frontend && npm run dev</code>
Backend	<code>http://localhost:5000</code>	<code>cd backend && ./venv/Scripts/python run.py</code>
Health check	<code>http://localhost:5000/api/health</code>	—
Run tests	—	<code>cd backend && ./venv/Scripts/python -m pytest tests/ -v</code>

Phase 14 — PostgreSQL (Production Database)

Manual steps:

- Sign up (free tier) at one of: neon.tech, supabase.com, or render.com.
- Create a new Postgres project/database.
- Copy the **connection string** — it looks like `postgresql://user:password@host:5432/dbname`.
- Send me that connection string in chat (plain text is fine — it only ever goes into your local, gitignored `.env` file, never committed).

Why: SQLite is a single file, fine for local development, but not safe for multiple concurrent server processes or guaranteed persistence on most cloud hosts. PostgreSQL is a real database server built for that.

Phase 15 — Deploy Flask Backend to Render

Manual steps:

- Sign up / log in at render.com, connect your GitHub account.
- Create a new **Web Service**, select the `v595/expenses` repo.
- Set **Root Directory** to `backend`.
- Set **Build Command** to `pip install -r requirements.txt`.

- Set **Start Command** to `gunicorn run:app` (we'll add `gunicorn` to `requirements.txt` when we get here).
- Add environment variables in Render's dashboard: `DATABASE_URL` (from Phase 14), `SECRET_KEY` (generate a strong random value — I can generate one for you), `FLASK_DEBUG=False`.
- Deploy, then send me the resulting backend URL (e.g. `https://your-app.onrender.com`).

Important: never enable Flask debug mode in production — it can expose an interactive code execution console to anyone who triggers an error.

Phase 16 — Deploy React Frontend to Vercel

Manual steps:

- Sign up / log in at **vercel.com**, connect your GitHub account.
- Import the `v595/expenses` repo as a new project.
- Set **Root Directory** to `frontend`.
- Add environment variable `VITE_API_URL` = your Render backend URL + `/api` (e.g. `https://your-app.onrender.com/api`).
- Deploy, then send me the resulting frontend URL (e.g. `https://your-app.vercel.app`).

Phase 17 — Connect Production Frontend + Backend

Once both URLs exist, I'll update the backend's CORS configuration to allow only your real Vercel URL (not a wildcard), then we'll test registration, login, and full CRUD against the live production stack together.

Phase 18 — Production Security Checklist

- Passwords hashed — already done (Phase 5).
- Strong, unique `SECRET_KEY` set in Render (not the local dev default).
- All secrets in environment variables — never committed to Git.
- HTTPS everywhere — Render and Vercel provide this automatically.
- CORS restricted to your real frontend origin only.
- Debug mode disabled in production.
- Users can only access their own data — already enforced (Phase 5).
- SQL injection protected via parameterized queries — already done throughout.

Environment Variables Reference

Variable	Where	Purpose
<code>DATABASE_URL</code>	Backend (Render)	Postgres connection string

SECRET_KEY	Backend (Render)	Flask secret, used for signing
FLASK_DEBUG	Backend (Render)	Must be False in production
VITE_API_URL	Frontend (Vercel)	Backend API base URL

Reminder: never paste passwords, API keys, or connection strings into a public place. Sharing them with me in this private chat is fine — they only ever end up in your local, gitignored .env files.